

Conditioning, Identifiability, and Regularization in an Adjoint-Based PDE Inverse Problem

Sipan Bareyan

June 29, 2026

Note on the use of artificial intelligence tools. Generative AI tools were used as an aid for writing and formatting this report (rephrasing, structuring, translation). This use was made with the agreement of my internship supervisor.

1 Goal

We study the inverse problem of recovering a potential V in a nonlinear PDE from a target state u_0 . The report asks when this problem is solvable. We compare several optimizers on a well-conditioned and an ill-conditioned instance, trace the ill-conditioned failure to the Hessian of the misfit (its conditioning and a null space in V), and show how regularization restores a usable recovery.

2 Problem setup

We consider the nonlinear PDE

$$-u'' + \alpha u^3 + Vu = f, \quad x \in [0, 2\pi], \quad u(0) = u(2\pi), \quad u'(0) = u'(2\pi),$$

where u is the state, V the potential to recover, α a nonlinearity parameter, and f a source term. We recover V from a target state u_0 by minimizing the misfit

$$M(V) = \int_0^{2\pi} (u(V) - u_0)^2 dx,$$

where $u(V)$ solves the PDE for V . We measure a recovery by the relative potential error $e(V) = \|V - V_\star\| / \|V_\star\|$.

We use two instances, both with true potential $V_\star = 3 + \cos x$. The well-conditioned one takes $u_{\text{well}} \propto 2 + \sin x$ (strictly positive); the ill-conditioned one takes $u_{\text{ill}} \propto \sin x$ (sign-changing, with zeros at $x = 0$ and $x = \pi$). Each target is normalized to $\|u_0\|_{L^2} = 1$. The data are exact: instead of a forward solve, we fix u_0 and back out the source from the PDE,

$$f = -u_0'' + \alpha u_0^3 + V_\star u_0,$$

so u_0 solves the PDE for $V_\star$ by construction. We then recover V and compare it to $V_\star$ (fig. 1).

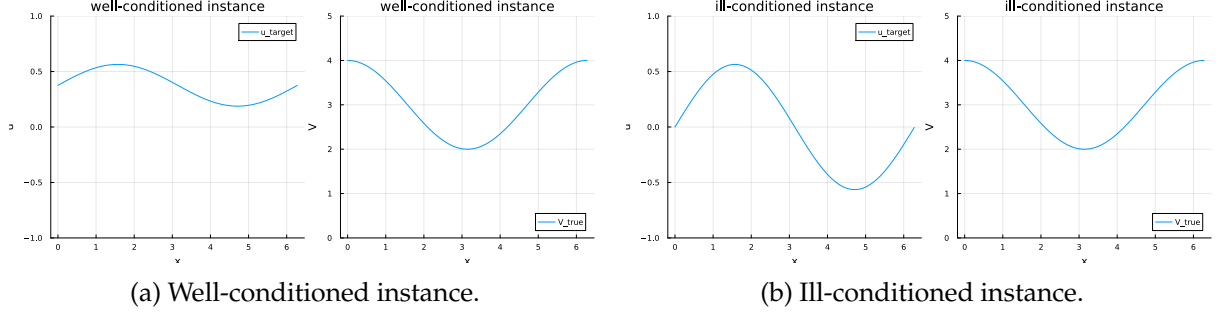


Figure 1: Target state u_0 and true potential V_* for the two instances used throughout.

3 Adjoint gradient

To run a gradient-based optimizer we need the gradient of $M(V)$. Report 1 derives it by the adjoint method, which returns the gradient with one extra linear solve and without differentiating $u(V)$ directly. In the experiments we compute gradients with Zygote.jl automatic differentiation and adjoint-based rules, the most efficient and general option we found.

4 Optimizer comparison: the identifiability gap

We compare three optimizers: gradient descent (GD), Adam, and L-BFGS. GD is the simplest. Adam adds adaptive step sizes and momentum; it is built for machine-learning problems but usually beats GD. L-BFGS is a quasi-Newton method that builds an inverse-Hessian approximation, which helps on ill-conditioned problems because it adapts to the local curvature.

We run all three on both instances. First we use a fixed iteration budget and record the loss and the potential error $e(V)$ at each iteration (fig. 2).

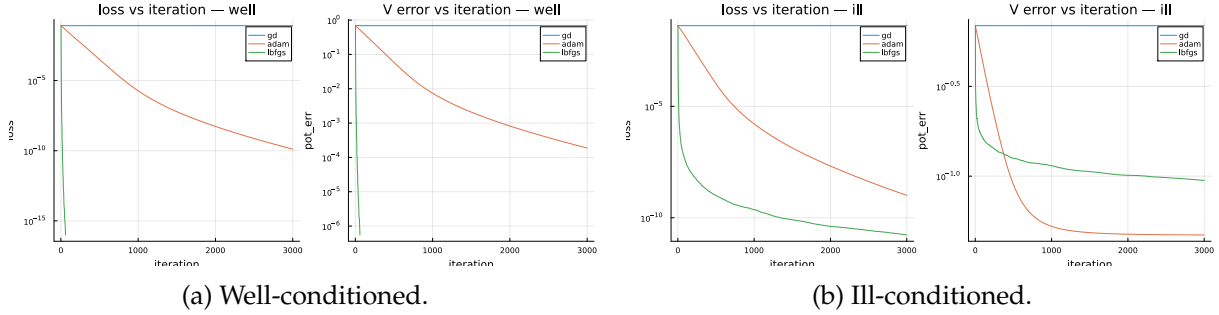


Figure 2: Loss and potential error $e(V)$ versus iteration, for the well-conditioned (left) and ill-conditioned (right) instances

Second we record the iterations and wall time each optimizer needs to reach a target loss, with the final loss and recovery error (table 1).

config	optimizer	it_to_target	s_to_target	it_to_tol	final_loss	pot_err	total_s
well	gd	-1	NaN	-1	0.0814066	0.687416	65.1595
well	adam	1076	17.796	-1	1.28919e-10	0.000187272	49.6171
well	lbfgs	8	0.185877	31	9.88558e-17	5.4984e-7	1.39408
ill	gd	-1	NaN	-1	0.0418023	0.687835	46.3377
ill	adam	1086	12.9499	-1	1.01487e-9	0.0470526	35.7731
ill	lbfgs	24	0.446273	-1	1.68581e-11	0.0946614	55.8028

Table 1: Iterations and wall time to the target/converged loss, final loss, and recovery, per optimizer.

On the well-conditioned instance the methods that move recover V . L-BFGS is fastest and reaches the lowest loss (8 iterations, under a second). Adam is slower and stops short of L-BFGS, reaching a potential error of about 2×10^{-4} at a loss near 10^{-10} . GD is the slowest and least efficient: its curves are almost flat (fig. 2). One fixed step size cannot move the stiff and the soft modes together, so the iterate is frozen.

The ill-conditioned instance shows the gap. The loss still falls to near machine zero, but the potential error stalls: minimizing the misfit no longer recovers V . The ranking even inverts. L-BFGS reaches the lower loss (about 2×10^{-11}) but the worse potential ($e(V) \approx 0.095$ in table 1); Adam settles at a higher loss (about 10^{-9}) but the better potential (≈ 0.047). For the ill-conditioned case we have problematic points: the zeros of u_0 at $x = 0, \pi$. The gradient of our loss w.r.t. V is a pointwise product of the state u with the adjoint variable, so it stays null wherever u vanishes; at convergence $u \approx u_0$, so V cannot be updated there. A lower loss does not mean a better recovery. The reason is structural, we can see in the next plot Figure 3 how the different optimizers shape differently the outputs: The lbfgs optimized V has oscillations around the problematic points, while Adam is more straightforward and forms sharp spikes at those points.

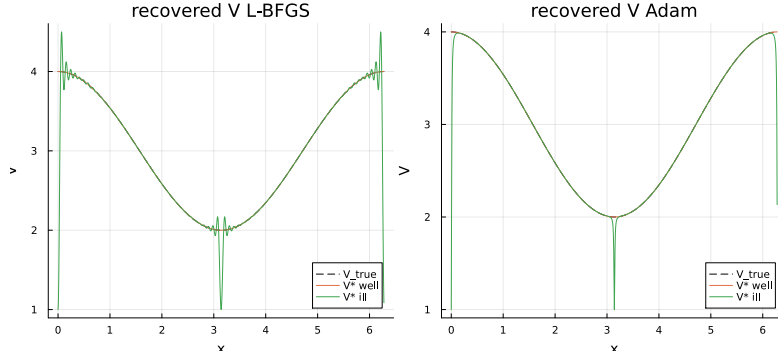


Figure 3: Recovered V : the ill-case error concentrates at the null points $x = 0, \pi$, where the data cannot constrain V .

5 The misfit Hessian: conditioning and identifiability

To analyze further the behaviour of optimization algorithms for this problem, we analyze the Hessian to identify complexity of the optimization.

5.1 Closed-form Hessian

From Report 1 we have the linearisation $L = -\partial^2 + \text{diag}(3\alpha u^2 + V)$ and the sensitivity $J = -L^{-1}\text{diag}(u)$. The misfit is

$$M(V) = \frac{w}{2} \|u(V) - u_0\|^2,$$

with residual $r = u - u_0$ and $w = 4\pi/N$, so that M is the trapezoidal form of the integral in section 2. By the adjoint method,

$$\nabla M = w J^\top r = -w \text{diag}(u) L^{-1} r = -w \text{diag}(u) \lambda,$$

where $\lambda = L^{-1}r$ is the adjoint state. Differentiate $\nabla M = w J^\top r$ using the product rule:

$$H \delta V = (\nabla \dot{M}) = w [(J^\top)^\dot{r} + J^\top \dot{r}].$$

The easy term: $\dot{r} = \delta u = J \delta V$, so

$$J^\top \dot{r} = J^\top J \delta V \quad (\text{Gauss-Newton term}).$$

The hard term uses $J^\top = -\text{diag}(u) L^{-1}$. By the product rule and the inverse rule on L^{-1} :

$$(J^\top)^\dot{r} = -\text{diag}(\delta u) L^{-1} + \text{diag}(u) L^{-1} \dot{L} L^{-1}.$$

Applying this to r and using $L^{-1}r = \lambda$:

$$(J^\top)^\dot{r} r = -\text{diag}(\delta u) \lambda + \text{diag}(u) L^{-1} \dot{L} \lambda.$$

Simplify each piece. For the first: $\text{diag}(\delta u) \lambda = \text{diag}(\lambda) \delta u = \text{diag}(\lambda) J \delta V$. For the second, expand $\dot{L} \lambda$ using $\dot{L} = \text{diag}(6\alpha u \odot \delta u + \delta V)$:

$$\dot{L} \lambda = 6\alpha \text{diag}(u \odot \lambda) \delta u + \text{diag}(\lambda) \delta V.$$

Then $\text{diag}(u) L^{-1} \dot{L} \lambda = -J^\top [6\alpha \text{diag}(u \odot \lambda) J \delta V + \text{diag}(\lambda) \delta V]$.

Combine both parts of the hard term:

$$(J^\top)^\dot{r} r = -\text{diag}(\lambda) J \delta V - 6\alpha J^\top \text{diag}(u \odot \lambda) J \delta V - J^\top \text{diag}(\lambda) \delta V.$$

Assemble the full Hessian by adding the Gauss-Newton term and the hard term:

$$H \delta V = w [J^\top J - 6\alpha J^\top \text{diag}(u \odot \lambda) J - J^\top \text{diag}(\lambda) - \text{diag}(\lambda) J] \delta V.$$

Thus

$$H = w (J^\top J + S), \quad S = -6\alpha J^\top \text{diag}(u \odot \lambda) J - J^\top \text{diag}(\lambda) - \text{diag}(\lambda) J.$$

The term $J^\top J$ is the Gauss-Newton component; every term in S carries λ and thus depends on the residual r . The Hessian is symmetric.

5.2 Conditioning and identifiability

We plot the computed Hessian in position space (fig. 4) and its spectrum (fig. 5). The Gauss-Newton component whose (i, j) entry is $\frac{4\pi}{N} u_i (L^{-2})_{ij} u_j$ explains the smoothing band and the loss of identifiability at zeros of u

For the well-conditioned instance ($u_{\text{well}} \propto 2 + \sin x > 0$) the factor $\text{diag}(u)$ is invertible, so the Gauss-Newton component has the full rank of L^{-2} and the Hessian spectrum decays smoothly. The decay still matters: L^{-2} damps high-frequency modes of V , so those directions are weakly constrained, but none is lost. Every direction in V is identifiable.

For the ill-conditioned instance the spectrum drops to machine zero, a null space. The cause is the factor u : $u_{\text{ill}} \propto \sin x$ vanishes at $x = 0$ and $x = \pi$, so the matching rows and columns of the Gauss–Newton component are zero, the dead cross-bands in fig. 4b. The same factor appears in the gradient $-\frac{4\pi}{N}(u \odot \lambda)$, which cannot move V where $u = 0$. The misfit is flat along these directions, so the data do not constrain V at the zeros of u , and the problem is non-identifiable there. This is exactly the failure seen in section 4: fig. 3 shows the recovered V matching V_* except at $x = 0, \pi$.

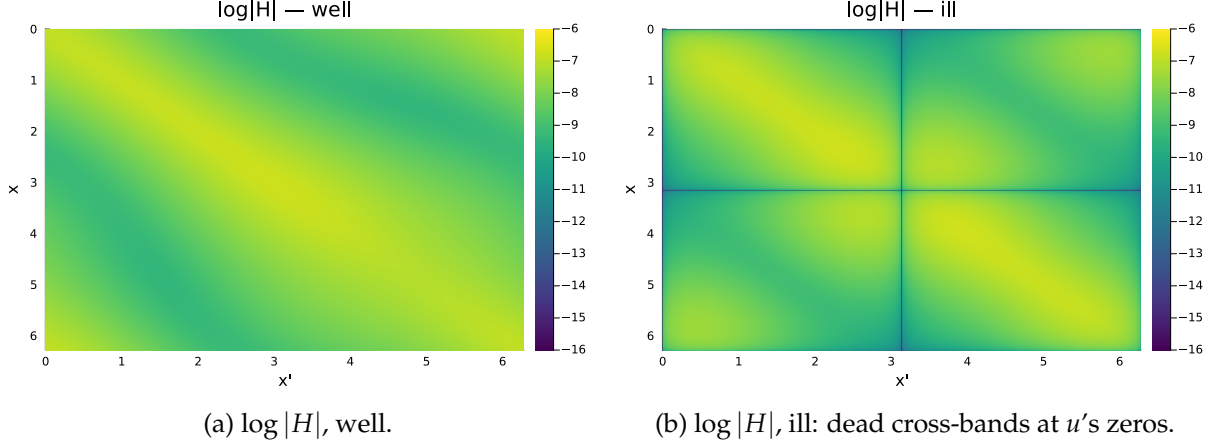


Figure 4: Hessian magnitude in position space

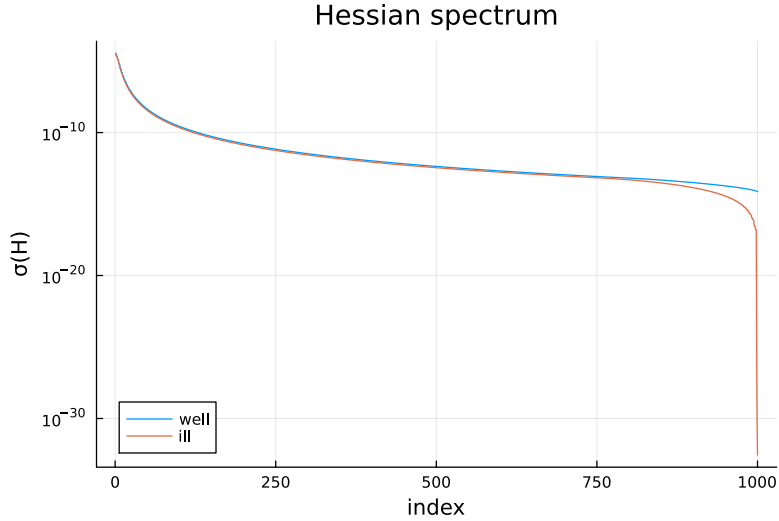


Figure 5: Hessian spectrum for both instances

config	κ	pot_err
well	4.71717e9	5.4984e-7
ill	1.21451e28	0.0866148

Table 2: Condition number κ and recovery error per instance.

5.3 Dependence on nonlinearity and resolution

The nonlinearity α and the resolution N change the problem without changing its kind. They act on different parts of the full Hessian. The smoothing and rank effects are already visible in

the Gauss–Newton component so we sweep them separately.

5.3.1 Nonlinearity α

α enters the full Hessian through $L = -\partial_{xx} + \text{diag}(3\alpha u^2 + V)$ and through the correction S . A larger α adds $3\alpha u^2$ to the diagonal of L , lifting its smallest eigenvalues (the smooth, low-frequency modes, where the $-\partial_{xx}$ term is weak), while the largest eigenvalue, set by $-\partial_{xx}$ at $O(N^2)$, is essentially unchanged. So $\kappa(L) = \lambda_{\max}/\lambda_{\min}$ shrinks as $\lambda_{\min}$ rises, and since H carries L^{-2} the conditioning of H drops with it. The sweep confirms this for the well case: κ falls about three orders of magnitude, from 3.7×10^{10} at $\alpha = 0.5$ to 1.6×10^7 at $\alpha = 500$ (fig. 6, table 3), while the Newton solve stiffens and needs more iterations. The ill case behaves differently: $\text{diag}(u)$ is singular at every α , so κ stays near 10^{28} and the recovery error stays around 0.08 to 0.10. Nonlinearity improves the conditioning of the well problem but does not remove the null space of the ill one.

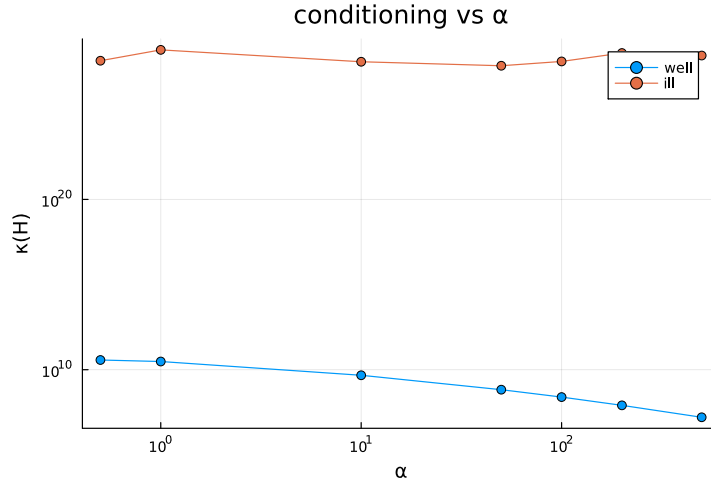


Figure 6: Condition number κ versus nonlinearity α . The well case falls about three orders of magnitude; the ill case stays near 10^{28} .

α	newton	κ	it_to_tol	pot_err	α	newton	κ	it_to_tol	pot_err
0.5	4	3.71531e10	47	4.82702e-7	0.5	4	1.39524e28	-1	0.0987093
1.0	4	3.02892e10	44	5.46582e-7	1.0	5	6.06061e28	-1	0.0968984
10.0	7	4.71717e9	31	5.4984e-7	10.0	6	1.21451e28	-1	0.0866148
50.0	10	6.71824e8	14	1.00638e-6	50.0	9	7.07676e27	3733	0.0769977
100.0	11	2.47071e8	12	1.25908e-6	100.0	10	1.26763e28	-1	0.0781383
200.0	13	8.07963e7	12	7.38907e-7	200.0	12	3.96139e28	-1	0.0807069
500.0	15	1.60956e7	14	1.07258e-6	500.0	15	2.81471e28	-1	0.0799705

α -sweep, well-conditioned
 α -sweep, ill-conditioned

Table 3: α -sweep conditioning data for both instances.

5.3.2 Resolution N

Resolution acts on L^{-2} . The eigenvalues of $L \approx -\partial_{xx}$ run from $O(1)$ for the smoothest mode to $O(N^2)$ for the highest mode the grid resolves, so those of L^{-2} run from $O(1)$ down to $O(N^{-4})$. For the well case $\text{diag}(u)$ is bounded and invertible and does not change this, so

$$\kappa(H) \sim N^4,$$

the straight line in fig. 7. A finer mesh exposes finer, less constrained modes of V . The ill case has κ near 10^{30} at every N , set by the null space and the round-off floor, with no clean trend.

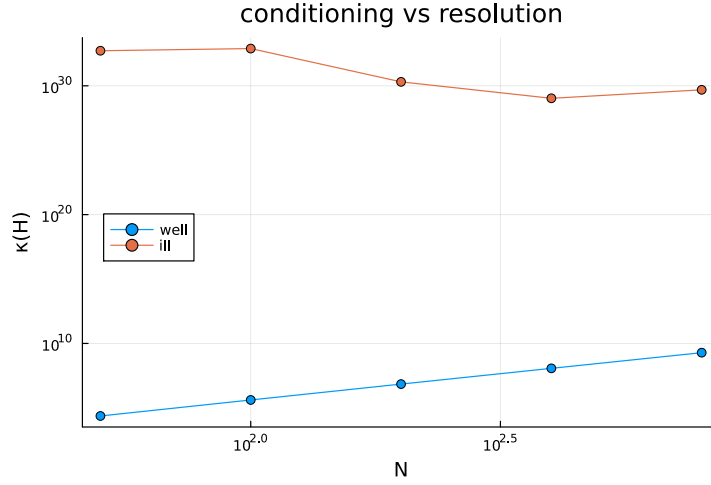


Figure 7: Condition number κ versus resolution N . The well case grows as N^4 ; the ill case stays high and irregular.

N	κ_{well}	κ_{ill}
50	23399.7	5.26401e32
100	410071.0	7.77475e32
200	6.99582e6	2.02716e30
400	1.16767e8	1.071e29
800	1.91974e9	4.87682e29

Table 4: N -sweep (resolution) conditioning data.

6 Regularization

Regularization helps in three ways. It stabilizes the inversion when u_0 is noisy; it adds prior knowledge about V , such as smoothness or sparsity; and it fills in the null directions of the Hessian, so V can be recovered where the data alone cannot constrain it.

Noise makes the need clear. We perturb u_0 by about 10^{-6} , too small to see (fig. 8, right), and try the pointwise inversion that solves the PDE for V directly,

$$V_{\text{direct}} = \frac{f + u'' - \alpha u^3}{u}.$$

This differentiates the noisy u twice, and the u'' term scales a Fourier mode of wavenumber k by k^2 , so the high-frequency noise is amplified and V_{direct} is too noisy to use (fig. 8, left). The variational recovery V^* never differentiates the data and stays close to V_* .

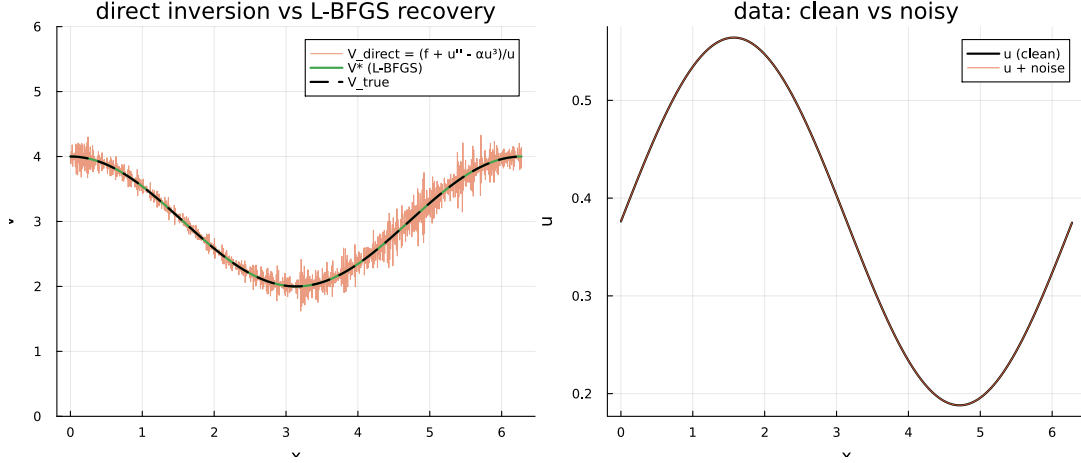


Figure 8: Left: the pointwise inversion $V_{\text{direct}} = (f + u'' - \alpha u^3)/u$ and L-BFGS recovery on noisy well-conditioned data Right: the added noise is invisible to the naked eye.

The penalty has to match the unknown. We compare two, each scaled by β and added to the misfit:

1. Tikhonov $R_2(V) = \beta \|V'\|_2^2$: favors smoothness
2. Total variation $R_1(V) = \beta \|V'\|_1$: favors piecewise-constant shapes with sharp edges

Which one wins depends on V_* . For a smooth V_* (a cosine) Tikhonov matches the prior and reaches about $85\times$ lower error than TV at the best β (fig. 9, table 5); at β^* it also removes the null-point spikes of section 4.

For a box V_* (a top-hat) the order flips: TV keeps the edges that Tikhonov smooths out (fig. 10). Regularization helps when the penalty matches the structure of the unknown, supplying what the data leave undetermined.

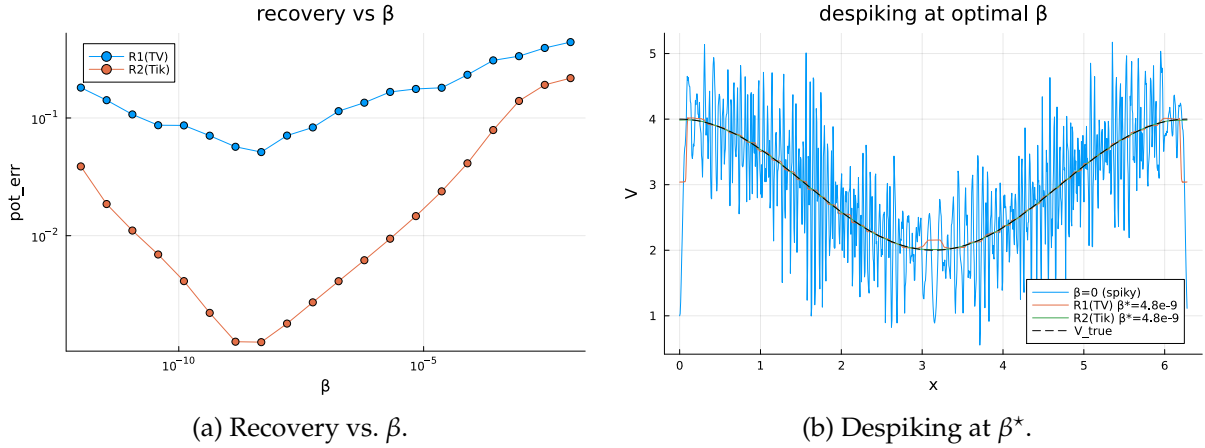


Figure 9: Smooth V_* (cosine): Tikhonov reaches the lower error and reconstructs the smooth potential.

method	β	misfit	pot_err
R1(TV)	4.83293e-9	1.05823e-8	0.0513282
R2(Tik)	4.83293e-9	9.74167e-9	0.00124614

smooth $V_\star$

method	β	misfit	pot_err
R1(TV)	1.27427e-10	9.31144e-9	0.0353655
R2(Tik)	3.35982e-12	8.96603e-9	0.0468207

box $V_\star$

Table 5: Best regularization weight $\beta^\star$ (the value minimizing the potential error) for smooth and box $V_\star$.

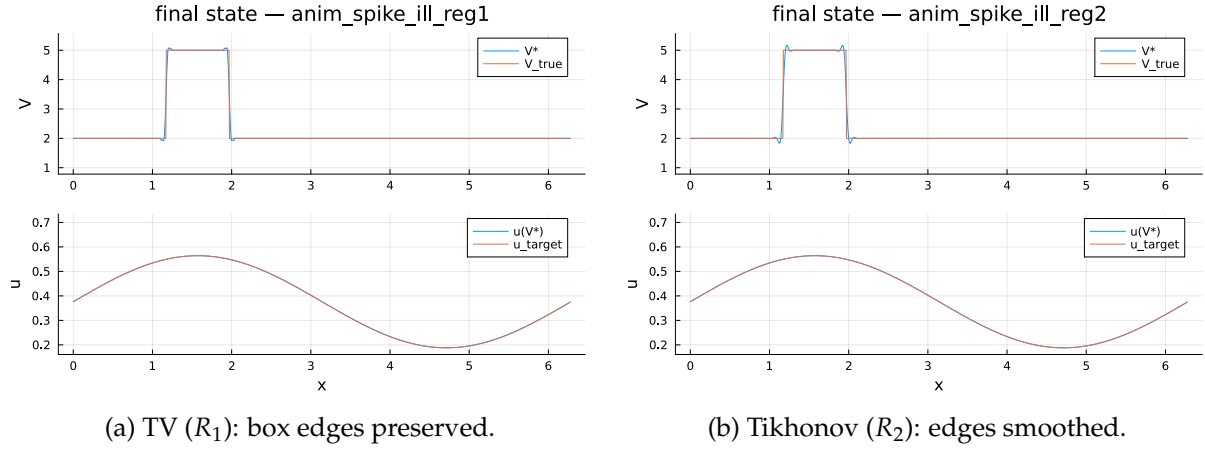


Figure 10: Box $V_\star$, converged $V^\star$ (top) and state $u(V^\star)$ (bottom): TV keeps the edges, Tikhonov smooths them.

A Reproducibility

13th Gen Intel(R) Core(TM) i7-1360P (16 threads); Julia 1.12.6, BLAS threads 8; commit 3dce43b, 2026-06-29.